The Transformer, from Tokens to Text

How a decoder-only, dense LLM works: the math, the diagrams, and runnable code

---

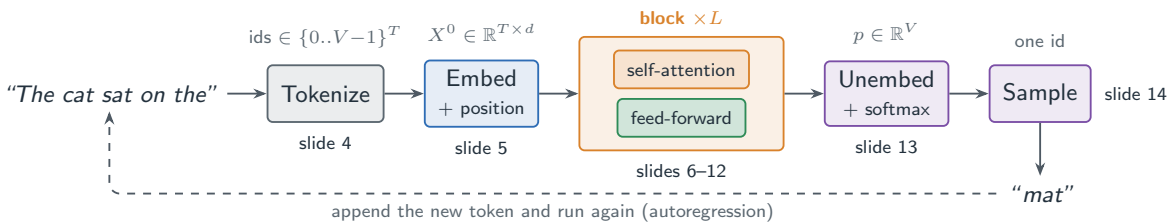
A 20-slide tutorial • companion code: `tokenizer.py`, `model.py`, `train.py`, `walkthrough.py`

The job: predict the next token

A language model assigns a probability to a token sequence. The chain rule turns that into a sequence of next-token predictions:

$$p(x_1, \ldots, x_T) \;=\; \prod_{t=1}^{T} p_\theta\big(x_t \mid x_1, \ldots, x_{t-1}\big).$$

- **Decoder-only**: one stack; each position sees only tokens to its left (GPT family [2,3]).
- **Dense**: every parameter is used for every token (no expert routing, as in MoE models).



Everything between "Tokenize" and "Sample" is one differentiable function f_θ; training (slides 15–16) adjusts θ so the true next token gets high probability.

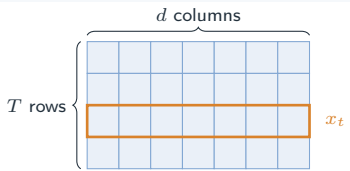
Notation: six numbers define the model

	meaning	**this tutorial**	GPT-2 small [3]
V	vocabulary size	512	50,257
$T_{\max}$	context window (tokens)	128	1,024
d	model width	128	768
h	attention heads	4	12
d_k	width of one head, d/h	32	64
L	number of blocks	4	12
	parameters	875,264	124 M

Same architecture, same equations: only these six numbers change between a laptop model and a production one.

Conventions

- A sequence is a matrix $X \in \mathbb{R}^{T \times d}$: **one row per token**.
- Vectors are rows, so a linear layer is xW with $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$.
- X^{ℓ} is the matrix after block ℓ; X^{0} is the embedding.
- x_t is row t: "what the model knows at position t".



Stage 1 • Tokenization: text becomes integers

A network needs numbers. Which units of text get a number?

- **Characters**: tiny vocabulary, very long sequences.
- **Words**: short sequences, but unseen words are unrepresentable.
- **Subwords**: common words are one token, rare words split into pieces.

Byte-pair encoding (BPE) [4]

1. Start with one token per byte (256 tokens).
2. Count every adjacent pair of tokens in the corpus.
3. Merge the most frequent pair into a new token.
4. Repeat 2–3 until the vocabulary has V entries.

To encode new text, replay the merges in the order learned. Decoding is concatenation, so tokenization is lossless.

Toy corpus (word:count) `low:5 lower:2 newest:6 widest:3`

merge	pair	new token (id)	count
1	`e + s`	`es` (256)	9
2	`es + t`	`est` (257)	9
3	`l + o`	`lo` (258)	7
4	`lo + w`	`low` (259)	7

`lowest` never occurs in the corpus, yet encodes as two known pieces:

`low` `est`
259 257

Real example: our $V{=}512$ tokenizer trained on Shakespeare

`T` `o` `␣be` `,` `␣` `or` `␣not` `␣to` `␣be`
84 111 307 44 32 270 325 288 307

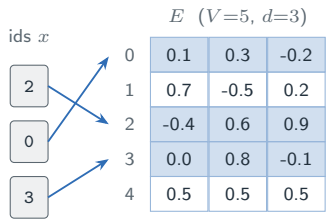
19 characters → 9 tokens; both ␣be map to id 307. Corpus average: 2.00 characters per token.

Stage 2 • Embedding: integers become vectors

An id is just a name; id 307 is not "bigger" than id 44. Give every token a **learned vector** instead:

$$x_t^0 = \underbrace{E[x_t]}_{\text{what}} + \underbrace{P[t]}_{\text{where}}, \qquad E \in \mathbb{R}^{V \times d}, \;\; P \in \mathbb{R}^{T_{\max} \times d}.$$

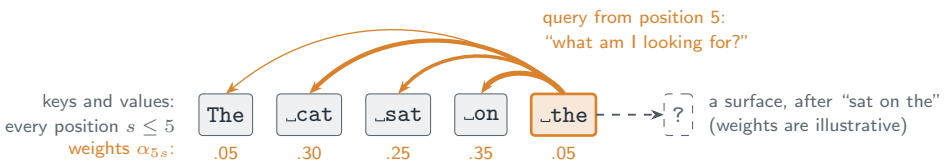
- $E[x_t]$ is **row x_t of a table**. Equivalently $\mathrm{onehot}(x_t)\, E$: a linear layer applied to a one-hot vector.
- Training moves tokens used in similar contexts toward similar vectors.
- **Why P?** Attention (next slides) treats its input as a *set*: without positions, "dog bites man" and "man bites dog" look identical.
- We learn P like GPT-2 [3]. The original paper used fixed sinusoids [1]; many recent models rotate queries and keys instead (RoPE [17]).



$$\underbrace{\begin{bmatrix} -0.4 & 0.6 & 0.9 \\ 0.1 & 0.3 & -0.2 \\ 0.0 & 0.8 & -0.1 \end{bmatrix}}_{E[x]} + \underbrace{\begin{bmatrix} 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.1 \\ 0.2 & -0.1 & 0.0 \end{bmatrix}}_{P} = \underbrace{\begin{bmatrix} -0.4 & 0.7 & 0.9 \\ 0.2 & 0.3 & -0.1 \\ 0.2 & 0.7 & -0.1 \end{bmatrix}}_{X^0}$$

Stage 3 • Self-attention: the idea

The embedding of "the" is the same in every sentence. To predict what follows, position t must **gather information from earlier positions**. Attention is a soft, learnable lookup [1,5]:



Each position t produces three vectors by multiplying its row x_t with three learned matrices:

Query $q_t = x_t W_Q$
What position t is looking for.

Key $k_s = x_s W_K$
What position s advertises about itself.

Value $v_s = x_s W_V$
What position s hands over if chosen.

$$\underbrace{s_{ts} = \frac{q_t \cdot k_s}{\sqrt{d_k}}}_{\text{1. score each earlier position}} \qquad \underbrace{\alpha_{ts} = \frac{e^{s_{ts}}}{\sum_{r \leq t} e^{s_{tr}}}}_{\text{2. normalise: weights sum to 1}} \qquad \underbrace{o_t = \sum_{s \leq t} \alpha_{ts}\, v_s}_{\text{3. weighted average of values}}$$

Stage 3 • Self-attention: the math in one line

$$\boxed{\mathrm{Attn}(X) \;=\; \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V} \qquad Q = XW_Q,\;\; K = XW_K,\;\; V = XW_V$$

Shapes. $X \in \mathbb{R}^{T\times d}$; $W_Q, W_K, W_V \in \mathbb{R}^{d\times d_k}$; so $Q, K, V \in \mathbb{R}^{T\times d_k}$. $QK^\top \in \mathbb{R}^{T\times T}$ holds every query–key dot product at once; softmax acts on each row.

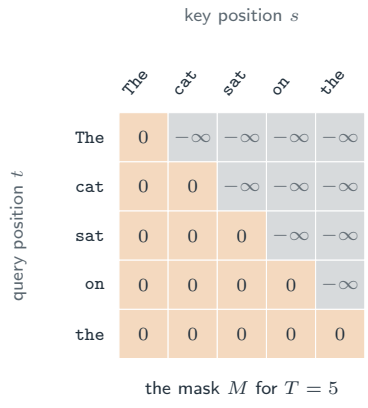
Why the causal mask M?

$$M_{ts} = \begin{cases} 0 & s \le t \\ -\infty & s > t \end{cases} \qquad \Rightarrow \qquad e^{-\infty} = 0.$$

Position t gets zero weight on the future. One forward pass therefore yields T honest next-token predictions, and training matches generation.

Why divide by $\sqrt{d_k}$? If entries of q, k have variance 1, then $q \cdot k$ has variance d_k. Large scores saturate softmax (one weight ≈ 1, gradients ≈ 0); rescaling keeps scores at unit scale for any head width [1].

Cost. T^2 scores per head: time and memory grow quadratically with context length.



the mask M for $T = 5$

Row t may look at columns $1..t$ only.

Stage 3 • Self-attention: a worked example ($T = 3$, $d_k = 2$)

Three tokens, one head. Follow the formula left to right (run `python walkthrough.py` to reproduce):

$$\underbrace{\begin{bmatrix}1 & 0\\ 0 & 2\\ 2 & 1\end{bmatrix}}_{Q}\underbrace{\begin{bmatrix}1 & 0 & 2\\ 1 & 2 & 0\end{bmatrix}}_{K^\top}\frac{1}{\sqrt{2}} = \underbrace{\begin{bmatrix}0.71 & 0 & 1.41\\ 1.41 & 2.83 & 0\\ 2.12 & 1.41 & 2.83\end{bmatrix}}_{\text{scores } S} \xrightarrow{+M} \begin{bmatrix}0.71 & -\infty & -\infty\\ 1.41 & 2.83 & -\infty\\ 2.12 & 1.41 & 2.83\end{bmatrix} \xrightarrow{\text{row softmax}} \underbrace{\begin{bmatrix}1 & 0 & 0\\ 0.196 & 0.804 & 0\\ 0.284 & 0.140 & 0.576\end{bmatrix}}_{\text{weights } A}$$

$$\text{output} = AV = A\underbrace{\begin{bmatrix}1 & 0\\ 0 & 1\\ 1 & 1\end{bmatrix}}_{V} = \begin{bmatrix}1 & 0\\ 0.196 & 0.804\\ 0.860 & 0.716\end{bmatrix}$$

Row 2 by hand. Scores $(1.41,\ 2.83)$:

$$\frac{e^{1.41}}{e^{1.41}+e^{2.83}} = \frac{4.11}{4.11+16.92} = 0.196, \qquad 1-0.196 = 0.804.$$

Its output is $0.196\,v_1 + 0.804\,v_2 = (0.196,\ 0.804)$.

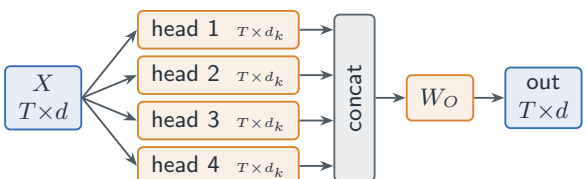
Read the result

- **Row 1** can only see itself: weight 1, output $= v_1$.
- **Row 2** prefers token 2 (higher score) but keeps 20% of token 1.
- **Row 3** sees all three; its output mixes all values.
- Every row of A is a probability distribution, and A is lower-triangular. Both are checked in `test_model.py`.

Stage 3 • Multi-head attention: several lookups in parallel

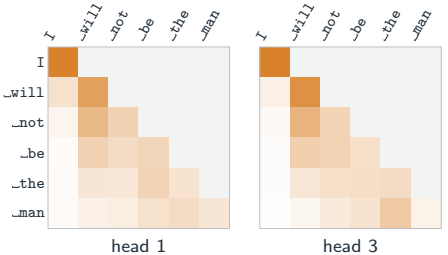
One head gives one weighted average per position. Language needs several relations at once (the subject, the previous word, the matching quote…), so run h heads side by side, each with its own $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d \times d_k}$, $d_k = d/h$:

$$\mathrm{MHA}(X) = \big[\mathrm{Attn}_1(X) \,\|\, \cdots \,\|\, \mathrm{Attn}_h(X)\big]\, W_O, \qquad W_O \in \mathbb{R}^{d \times d}.$$



- Splitting d into h pieces keeps the cost of one wide head.
- W_O mixes the heads back into the shared d-dimensional space.
- In code this is one reshape: $(B, T, d) \to (B, h, T, d_k)$.

Two heads of our trained model, block 1



Row = query token, column = key token; darker = larger weight.

Measured from `out/ckpt.pt`; the upper triangle is exactly zero.

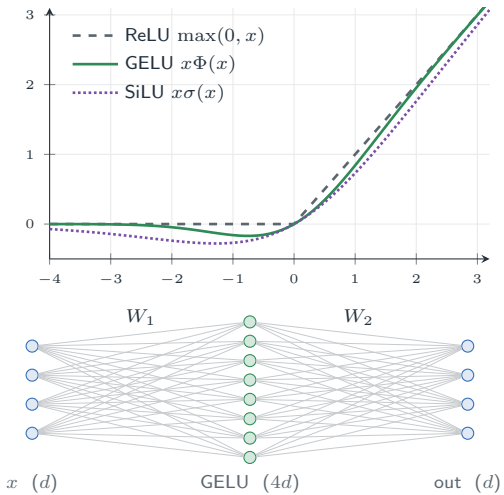
Stage 4 • Feed-forward network and the activation function

Attention *moves* information between positions. The feed-forward network (FFN) *transforms* it, at every position independently, with the same weights:

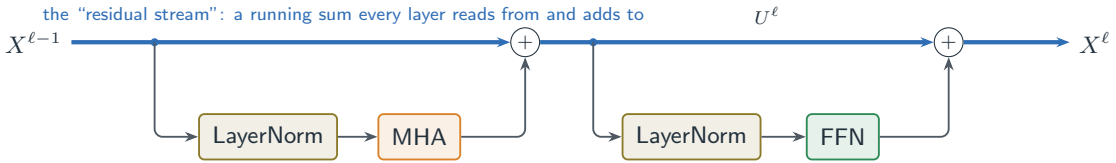
$$\mathrm{FFN}(x) = \mathrm{GELU}(xW_1 + b_1)\,W_2 + b_2,$$

$$W_1 \in \mathbb{R}^{d\times 4d}, \qquad W_2 \in \mathbb{R}^{4d\times d}.$$

- **Expand, bend, project**: $d \to 4d \to d$. The wide hidden layer is where most parameters live: $\approx \frac{2}{3}$ of each block.
- **Why a nonlinearity?** Without it, W_1W_2 is one matrix and the whole stack of layers could only compute linear functions of its input.
- **Which one?** $\mathrm{GELU}(x) = x\,\Phi(x)$, with Φ the standard normal CDF [8]: a smooth ReLU with nonzero gradient for $x < 0$. Used by GPT-2 and BERT. Recent models often use the related SiLU inside a gated FFN (SwiGLU [18]).



Stage 5 • Residual connections and LayerNorm: one block



$$U^{\ell} = X^{\ell-1} + \mathrm{MHA}\big(\mathrm{LN}(X^{\ell-1})\big), \qquad X^{\ell} = U^{\ell} + \mathrm{FFN}\big(\mathrm{LN}(U^{\ell})\big)$$

Residual connection [7]

Each sublayer computes an *update* that is added to its input.

- The $+$ passes gradients straight through, so even the first block gets a clean training signal in a deep stack.
- At initialisation a block is close to the identity, a safe starting point.

Layer normalisation [6]

Rescale each token's vector to mean 0, variance 1, then apply a learned gain and bias:

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \epsilon}} + \beta, \qquad \gamma, \beta \in \mathbb{R}^d.$$

Example: $\mathrm{LN}([2, 4, 6, 8]) = [-1.34, -0.45, 0.45, 1.34]$.

Pre-LN (shown; GPT-2 and later) normalises the *input* of each sublayer and keeps the stream itself untouched. The original transformer normalised after the addition; pre-LN trains more stably [13].

The whole model on one slide

Forward pass, for token ids $x_{1:T}$:

$$
\begin{aligned}
X^0 &= E[x_{1:T}] + P_{1:T} && \text{embed}\\
U^\ell &= X^{\ell-1} + \mathrm{MHA}^\ell\big(\mathrm{LN}(X^{\ell-1})\big) && \text{communicate}\\
X^\ell &= U^\ell + \mathrm{FFN}^\ell\big(\mathrm{LN}(U^\ell)\big), \quad \ell = 1..L && \text{compute}\\
Z &= \mathrm{LN}(X^L)\, E^\top && \text{unembed}\\
p_\theta(\,\cdot \mid x_{\le t}) &= \mathrm{softmax}(Z_t) && \text{predict}
\end{aligned}
$$

with the pieces

$$
\mathrm{Attn}_i(X) = \mathrm{softmax}\Big(\frac{(XW_Q^i)(XW_K^i)^\top}{\sqrt{d_k}} + M\Big) XW_V^i
$$

$$
\mathrm{MHA}(X) = \big[\mathrm{Attn}_1(X) \,\|\, \cdots \,\|\, \mathrm{Attn}_h(X)\big] W_O
$$

$$
\mathrm{FFN}(X) = \mathrm{GELU}(XW_1 + b_1)\, W_2 + b_2 \qquad \mathrm{LN}(x) = \gamma \odot \tfrac{x-\mu(x)}{\sigma(x)} + \beta
$$

Training chooses $\theta = \{E, P, W_\bullet^\ell, b_\bullet^\ell, \gamma_\bullet^\ell, \beta_\bullet^\ell\}$ to minimise

$$
\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta\big(x_{t+1} \mid x_{\le t}\big)
$$

Where the 875,264 parameters are

component	formula	count
E (tokens)	Vd	65,536
P (positions)	$T_{\max}d$	16,384
attention	$4d^2 + 4d$	66,048
FFN	$8d^2 + 5d$	131,712
2 LayerNorms	$4d$	512
one block		198,272
$\times L = 4$		793,088
final LayerNorm	$2d$	256
unembedding	tied to E	0

Every operation is a matrix product, an addition, a normalisation, or an element-wise function: all differentiable, so one call to backpropagation gives $\nabla_\theta \mathcal{L}$.

A fuller formal treatment: [15].

Stage 6 • From vectors back to tokens: unembedding and softmax

The final row $h_t = \mathrm{LN}(x_t^L) \in \mathbb{R}^d$ summarises the context. Score every vocabulary entry with a dot product:

$$z = h_t\, E^\top \in \mathbb{R}^V, \qquad z_j = h_t \cdot e_j.$$

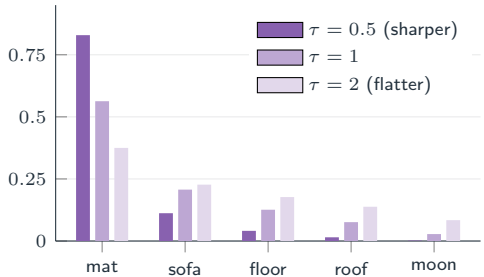
- z are the **logits**: token j scores high when its embedding e_j points the same way as h_t.
- **Weight tying** [12]: reuse the embedding table E as the output matrix. Fewer parameters, one shared meaning space.

Softmax turns scores into probabilities:

$$p_j = \mathrm{softmax}(z/\tau)_j = \frac{e^{z_j/\tau}}{\sum_{i=1}^{V} e^{z_i/\tau}}$$

- Positive, sums to 1, preserves order.
- Adding a constant to all z_j changes nothing: only *differences* matter.
- **Temperature** τ: $\tau \to 0$ picks the argmax; $\tau \to \infty$ is uniform.

"The cat sat on the ____" logits $z = (2,\ 1,\ 0.5,\ 0,\ -1)$

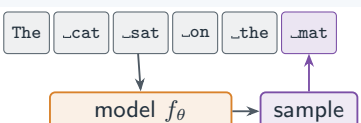


	mat	sofa	floor	roof	moon
$\tau = 0.5$	.829	.112	.041	.015	.002
$\tau = 1$	.563	.207	.126	.076	.028
$\tau = 2$	.375	.227	.177	.138	.084

Stage 7 • Generating text: choose a token, append, repeat

The generation loop

1. Run the model on the tokens so far (at most the last $T_{\max}$).
2. Take the logits of the **last** position only.
3. Turn them into a distribution and **pick one token**.
4. Append it. Go to 1.



The model is deterministic; all randomness enters at step 3. Production systems cache each layer's keys and values so that a step costs one new row, not a full re-run.

Step 3: decoding strategies on $p = (.563,\ .207,\ .126,\ .076,\ .028)$

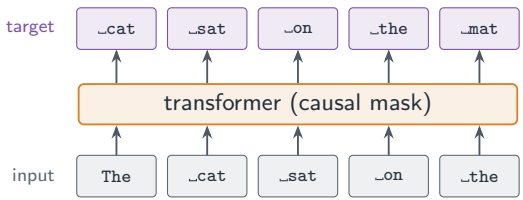
strategy	rule	result here
Greedy	take $\arg\max_j p_j$	always "mat"
Temperature	sample from $\mathrm{softmax}(z/\tau)$	τ=0.5: mat 83%
Top-k	keep the k most likely, renormalise, sample	k=2: (.731, .269)
Top-p [14]	keep the smallest set with total mass $\geq p$, renormalise, sample	p=0.9: drops "moon"

- Greedy is repeatable but tends to loop and repeat itself [14].
- Pure sampling ($\tau = 1$) is diverse, but the long tail of unlikely tokens occasionally derails the text.
- Top-k and top-p cut the tail and then sample: the usual compromise. Top-p adapts: few candidates when the model is sure, many when it is not.

Top-p check: cumulative mass $(.563,\ .770,\ .896,\ .972,\ 1)$ first reaches 0.9 at the 4th token.

Training • The objective: make the true next token likely

Training data is just text. Slide a window over it; the **target is the input shifted by one token**:



Because of the mask, one sequence of length T gives T training examples in a single pass.

Cross-entropy loss (average negative log-likelihood):

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta\left(x_{t+1} \mid x_{\le t}\right)$$

Assign probability 1 to the truth: loss 0. Assign it 0.207: loss $-\ln 0.207 = 1.57$.

The gradient is remarkably simple

For logits z, probabilities $p = \mathrm{softmax}(z)$ and one-hot target y:

$$\frac{\partial \mathcal{L}}{\partial z} = p - y.$$

Push the correct logit up by $1 - p_y$, push every other logit down by its own probability. Backpropagation carries this signal to every weight.

Example (target "sofa"):
$p - y = (.563,\ -.793,\ .126,\ .076,\ .028)$; confirmed against finite differences in `walkthrough.py`.

Two sanity numbers

- An untrained model guesses uniformly, so its loss should be $\ln V = \ln 512 = 6.24$. Ours started at 6.26.
- **Perplexity** $= e^{\mathcal{L}}$: the effective number of equally likely choices the model is hesitating between.

Training • The optimizer: AdamW with warmup and cosine decay

Plain gradient descent is $\theta \leftarrow \theta - \eta\, g$ with $g = \nabla_\theta \mathcal{L}$ on a random mini-batch. Transformers are trained with **AdamW** [9,10], which keeps two running averages per parameter:

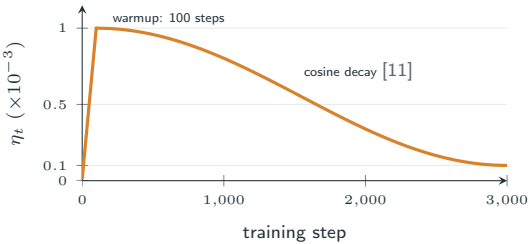
$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \qquad \text{mean of gradients}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2 \qquad \text{mean of squares}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \text{bias correction}$$

$$\theta_t = \theta_{t-1} - \eta_t \Big(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \theta_{t-1} \Big)$$

- **Momentum** (m) smooths noisy gradients.
- **Per-parameter scaling** ($\sqrt{v}$): each step is about η in size whatever the gradient's scale. On step 1 it is exactly $\pm\eta$.
- **Decoupled weight decay** (λ): shrink weight matrices a little every step to regularise; not applied to biases or LayerNorm gains.



Warmup: early on v_t is a poor estimate and the weights are random, so start with small steps. **Decay**: finish with small steps to settle into a minimum. **Gradient clipping** [16]: if $\|g\|_2 > 1$, rescale g to norm 1 so one bad batch cannot wreck the weights.

Settings in `train.py`			
peak η	10^{-3}	β_1, β_2	0.9, 0.95
final η	10^{-4}	λ	0.1
batch $\times$ T	32×128	clip norm	1.0

Code • The model: each equation is a line of PyTorch (`model.py`)

```python
class CausalSelfAttention(nn.Module):
    """A = softmax(Q K^T / sqrt(d_k) + M),   output = (A V) W_O."""
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, d = x.shape
        q, k, v = self.qkv(x).split(d, dim=-1)                          # each (B, T, d)
        # split d into h heads of width d_k:  (B, T, d) -> (B, h, T, d_k)
        q, k, v = (t.view(B, T, self.h, self.d_k).transpose(1, 2) for t in (q, k, v))
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.d_k)          # (B, h, T, T)
        scores = scores + self.mask[:T, :T]                             # causal mask
        attn = F.softmax(scores, dim=-1)                                # rows sum to 1
        out = self.attn_drop(attn) @ v                                  # (B, h, T, d_k)
        out = out.transpose(1, 2).reshape(B, T, d)                      # concatenate heads
        return self.resid_drop(self.proj(out))
class FeedForward(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.drop(self.fc2(F.gelu(self.fc1(x))))
class Block(nn.Module):
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
class GPT(nn.Module):
    def forward(self, idx: torch.Tensor, targets: torch.Tensor | None = None):
        pos = torch.arange(T, device=idx.device)
        x = self.drop(self.tok_emb(idx) + self.pos_emb(pos))       # X^0 = E[x] + P
        for block in self.blocks:
            x = block(x)
        logits = self.head(self.ln_f(x))                            # Z = LN(X^L) E^T
```

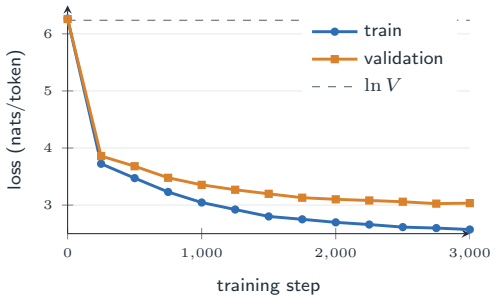
Lines included verbatim from `model.py` (constructors omitted). Compare with slides 7, 9, 10, 11 and 12.

Code • The training harness (`train.py`)

```python
def lr_at(step: int, args: argparse.Namespace) -> float:
    """Linear warmup to args.lr, then cosine decay to args.min_lr."""
    if step < args.warmup_steps:
        return args.lr * (step + 1) / args.warmup_steps
    progress = (step - args.warmup_steps) / max(1, args.max_steps - args.warmup_steps)
    return args.min_lr + 0.5 * (args.lr - args.min_lr) * (1 + math.cos(math.pi * progress))
def get_batch(data: torch.Tensor, block_size: int, batch_size: int, device: str):
    """x = tokens[i : i+T],  y = tokens[i+1 : i+T+1]  for random offsets i."""
    ix = torch.randint(len(data) - block_size - 1, (batch_size,))
    x = torch.stack([data[i:i + block_size] for i in ix])
    y = torch.stack([data[i + 1:i + block_size + 1] for i in ix])
    return x.to(device), y.to(device)
    for step in range(args.max_steps + 1):
        for g in opt.param_groups:                      # set this step's learning rate
            g["lr"] = lr_at(step, args)
        x, y = get_batch(splits["train"], args.block_size, args.batch_size, args.device)
        _, loss = model(x, y)                           # forward pass
        opt.zero_grad(set_to_none=True)
        loss.backward()                                 # backpropagation
        torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
        opt.step()                                      # AdamW update
```

`python train.py` trains the tokenizer, encodes the corpus (90/10 train/validation split), runs the loop above (evaluation and checkpointing lines omitted), logs to `out/log.csv`, saves the best checkpoint by validation loss, and prints a sample. Every hyperparameter is a flag: `--n_layer 6 --n_embd 384 --block_size 256` gives a GPU-sized model.

Results • What the harness produces



3,000 steps in 16 minutes on 2 CPU cores. Validation loss falls from 6.26 ($\approx \ln V$) to 3.03 (train 2.60): perplexity 21, down from 512. The train–validation gap is the model starting to memorise its 1 MB corpus.

Sample (prompt: newline; $\tau = 0.8$, top-$k = 50$)

```
wronger. Angelo, indeed,
In she time, I leave you all his house.

CLARENCE:
Good heavy.

ESTER:
Sir, it is it, that I do might.
```

Shakespeare-shaped, not Shakespeare: with under a million parameters the model has learned spelling, speaker labels and line structure, but little meaning. Scale d, L, data and compute, and the same code becomes a real LLM.

Try it

- `python test_model.py`: initial loss $\approx \ln V$, causality, overfit one batch.
- `python walkthrough.py`: every worked number in these slides.
- Exercises: remove the mask and watch validation loss; set $h = 1$; swap GELU for ReLU; train with `--tokenizer char`.

References

[1] Vaswani et al. *Attention Is All You Need.* NeurIPS 2017. arXiv:1706.03762

[2] Radford et al. *Improving Language Understanding by Generative Pre-Training.* OpenAI, 2018. (GPT)

[3] Radford et al. *Language Models are Unsupervised Multitask Learners.* OpenAI, 2019. (GPT-2)

[4] Sennrich, Haddow, Birch. *Neural Machine Translation of Rare Words with Subword Units.* ACL 2016. arXiv:1508.07909

[5] Bahdanau, Cho, Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate.* ICLR 2015. arXiv:1409.0473

[6] Ba, Kiros, Hinton. *Layer Normalization.* 2016. arXiv:1607.06450

[7] He, Zhang, Ren, Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016. arXiv:1512.03385

[8] Hendrycks, Gimpel. *Gaussian Error Linear Units (GELUs).* 2016. arXiv:1606.08415

[9] Kingma, Ba. *Adam: A Method for Stochastic Optimization.* ICLR 2015. arXiv:1412.6980

[10] Loshchilov, Hutter. *Decoupled Weight Decay Regularization.* ICLR 2019. arXiv:1711.05101

[11] Loshchilov, Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts.* ICLR 2017. arXiv:1608.03983

[12] Press, Wolf. *Using the Output Embedding to Improve Language Models.* EACL 2017. arXiv:1608.05859

[13] Xiong et al. *On Layer Normalization in the Transformer Architecture.* ICML 2020. arXiv:2002.04745

[14] Holtzman et al. *The Curious Case of Neural Text Degeneration.* ICLR 2020. arXiv:1904.09751

[15] Phuong, Hutter. *Formal Algorithms for Transformers.* 2022. arXiv:2207.09238

[16] Pascanu, Mikolov, Bengio. *On the Difficulty of Training Recurrent Neural Networks.* ICML 2013. arXiv:1211.5063

[17] Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* 2021. arXiv:2104.09864

[18] Shazeer. *GLU Variants Improve Transformer.* 2020. arXiv:2002.05202

[19] Karpathy. *nanoGPT* (code) and the Tiny Shakespeare corpus (*char-rnn*). github.com/karpathy

Further reading. [15] gives complete pseudocode for every transformer variant. [1] is the original (encoder–decoder, post-LN, sinusoidal positions); [3] is the decoder-only, pre-LN, GELU design implemented here.

Reproducibility. All numbers on slides 4–5, 8 and 13–16 are printed by `walkthrough.py`; slides 9 and 19 are generated from the training run by `make_figures.py`.